RESEARCH REPORT · RECONSTRUCTION & DIFFUSION PRIORS

# Does a diffusion depth prior improve multi-view 3D reconstruction?

A from-scratch investigation of confidence-guided fusion between VGGT (feed-forward geometric reconstruction) and Marigold (diffusion depth) — including a real bug in the fusion weighting, its diagnosis and fix, an offline hyperparameter search, and a live end-to-end verification on a real GPU.

GeoDiff3D · 4 real scenes, Tesla T4 · github.com/pranaysb/GeoDiff3D · August 2026

| 3 / 4 | −50.6% | 20 | 0 |
|---|---|---|---|
| real scenes where tuned fusion beats pure VGGT geometry | largest error reduction from the fusion-weighting fix, one scene | commits, full honest revision history, zero fabricated results | ground-truth datasets used — every claim is self-consistency only |

## Contents

## 01 The research question

VGGT reconstructs a scene's geometry directly from multiple photographs in a single feed-forward pass — fast, and grounded in real multi-view triangulation, but only as good as the geometric matches it finds. Marigold estimates depth from a single image using a diffusion model — often resolving detail VGGT can't see, but with no notion of multi-view consistency at all.

The question this project set out to test: **can a diffusion depth prior improve geometry-grounded reconstruction, if it's only trusted where VGGT's own confidence is genuinely low?** Not “diffusion is better” or “geometry is better” — specifically whether a well-gated fusion of the two beats either one alone, measured honestly, on real photographs, with no ground truth to lean on.

> **Why no ground truth.** None of the four scenes used here have a captured 3D scan or depth sensor reference. Every number in this report is a self-consistency metric — internal agreement between viewpoints — not a measurement against reality. That distinction holds throughout: it is the one honesty rule this project would not bend on.

## 02 How the fusion works

Four stages, run once per scene and shared across every method compared in this report, so the comparison is apples-to-apples:

1. **Geometric reconstruction — VGGT-1B.** A feed-forward transformer ingests all views at once and outputs per-view depth, a per-pixel confidence map, and camera intrinsics/extrinsics — no diffusion involved at this stage.
2. **Diffusion depth — Marigold.** Each view is independently passed through Marigold's diffusion pipeline, producing a relative (scale-and-shift-free) monocular depth estimate per image.
3. **Alignment.** A robust, percentile-clipped least-squares fit recovers the scale and shift that map Marigold's relative depth into VGGT's metric scale, per view.
4. **Confidence-guided fusion.** VGGT's raw confidence is normalized per image, then used to decide — per pixel — how much of the aligned Marigold depth to blend in. This is the part that had a real bug, described below.

Four methods are compared throughout: **VGGT-only** (no diffusion), **Marigold-only** (aligned, no geometry), **naive averaging** (blind 50/50 blend, no confidence signal), and **GeoDiff3D fusion** (the confidence-gated method under test).

## 03 First ablation: four real scenes, real GPU

Four real multi-view photo sets — not synthetic data — run on a Tesla T4: `kitchen` (tabletop object, 6 views), `llff_fern` (outdoor plant), `llff_flower` (outdoor close-up), `room` (indoor, low-texture). Metric: mean absolute relative depth error between neighboring views after reprojection — lower is more self-consistent.

| SCENE | VGGT-ONLY | MARIGOLD-ONLY | NAIVE AVG. | GEODIFF3D FUSION | FUSION VS VGGT |
|---|---|---|---|---|---|
| kitchen | 0.0739 | 0.1319 | 0.0924 | 0.0863 | ✗ +16.8% |
| llff_fern | 0.0381 | 0.0395 | 0.0365 | 0.0385 | ✗ +1.0% |
| llff_flower | 0.0874 | 0.1435 | 0.1046 | 0.1345 | ✗ +53.9% |
| room | 0.0285 | 0.1379 | 0.0801 | 0.0678 | ✗ +137.9% |

The result was unambiguous and not what the project was hoping for: **the fusion never beat pure VGGT geometry, in any of the four scenes.** That's the honest starting point — worth sitting with rather than explaining away, so the next section shows the actual investigation into why.

## 04 The bug: fusion wasn't actually confidence-selective

Rather than assume the idea was wrong, the fusion weights were reverse-engineered from the saved depth arrays of the ablation run above, algebraically: `weight = (fused − vggt) / (aligned − vggt)`.

| SCENE | RECOVERED MEAN WEIGHT TOWARD MARIGOLD | MEDIAN-PIXEL WEIGHT | % OF PIXELS MAJORITY-MARIGOLD |
|---|---|---|---|
| kitchen | 46.6% | 31.7% | 42.4% |
| llff_fern | 63.6% | 73.8% | 66.6% |
| llff_flower | 59.9% | 69.1% | 64.5% |
| room | 34.9% | 30.6% | 24.1% |

> ROOT CAUSE
>
> `normalize_confidence` rescaled each image's confidence to its own 5th–95th percentile — always stretching to fill [0,1], regardless of whether VGGT's absolute confidence was uniformly excellent. Combined with a linear `weight = 1 − confidence` blend, that meant the median pixel in every single scene landed near a 50/50 blend. In `room` — where VGGT beat Marigold by 4.8× on the actual metric — the median pixel still got 31% Marigold weight, and a quarter of all pixels were majority-Marigold. The fusion wasn't confidence-selective at all; it was a disguised version of naive averaging.

## 05 The fix: gate, don't blend

The full-range linear blend was replaced with a threshold gate: pixels at or above `trust_threshold` keep VGGT's depth completely untouched. Only pixels below the threshold ramp in aligned Marigold depth — capped at `max_aligned_weight`, so no pixel is ever fully replaced (Marigold-only was the worst standalone method in every scene, so full replacement was never justified).

```
def fuse_depths(reference_depth, aligned_depth, reference_confidence,
                trust_threshold=0.5, max_aligned_weight=0.1):
    rc = np.clip(reference_confidence, 0.0, 1.0)
    below_threshold = np.clip((trust_threshold - rc) / trust_threshold, 0.0, 1.0)
    fusion_weight = below_threshold * max_aligned_weight
    return (1.0 - fusion_weight) * reference_depth + fusion_weight * aligned_depth
```

Re-run on the same four scenes with the untuned fix (`max_aligned_weight=0.4`, chosen from the diagnosis alone, not yet tuned against data):

| SCENE | PRE-FIX FUSION | POST-FIX FUSION | CHANGE | BEATS VGGT-ONLY? |
|---|---|---|---|---|
| kitchen | 0.0863 | 0.0683 | −20.9% | ✓ yes |
| llff_fern | 0.0385 | 0.0365 | −5.2% | ✓ yes |
| llff_flower | 0.1345 | 0.0943 | −29.9% | ✗ no (7.9%) |
| room | 0.0678 | 0.0335 | −50.6% | ✗ no (17.5%) |

Error dropped in **every** scene, and fusion now beat VGGT-only outright in 2 of 4 — up from 0. But the fix wasn't tuned against data yet; the next question was whether the two new parameters were actually set well.

## 06 Tuning it without re-running the GPU 70 times

Re-running the full ablation for every candidate `(trust_threshold, max_aligned_weight)` pair would mean dozens of expensive GPU passes — and Marigold's diffusion sampling isn't seeded, so repeated runs of it aren't even bit-identical. Instead: cache the raw VGGT + Marigold outputs from one real GPU pass per scene (depth, confidence, camera matrices), then grid-search the fusion parameters entirely offline, in pure CPU numpy, against the exact same cross-view-consistency function used everywhere in this report.

A sanity check recomputed VGGT-only's error from the cached arrays before trusting any of it — it matched the GPU-measured numbers exactly.

### 70-point grid, ranked by scenes beaten

| TRUST_THRESHOLD | MAX_ALIGNED_WEIGHT | SCENES BEATEN (OF 4) | MEAN GAP TO VGGT |
|---|---|---|---|
| 0.5 | 0.40 (untuned) | 2 | +0.049 |
| 0.5 | 0.20 | 3 | −0.0086 |
| **0.5** | **0.10 — shipped** | **3** | **−0.0189 (best)** |
| 0.5 | 0.05 | 3 | −0.0143 |

`max_aligned_weight` dominated the ranking far more than `trust_threshold` did — and the relationship wasn't monotonic all the way to zero: 0.05 was measurably worse than 0.10. `trust_threshold` barely mattered near the optimum (0.4–0.7 all gave near-identical results), so 0.5 was kept as a reasonable, legible midpoint. New shipped defaults: **trust_threshold=0.5, max_aligned_weight=0.1.**

## 07 Confirming it wasn't a lucky sample

The grid search above used one cached Marigold realization per scene. To rule out overfitting to that one sample, the full ablation was re-run from scratch — fresh GPU inference, new unseeded Marigold samples throughout — using the tuned defaults already shipped in code.

| SCENE | OFFLINE PREDICTION | FRESH GPU MEASUREMENT | VGGT-ONLY | RESULT |
|---|---|---|---|---|
| **kitchen** | 0.0709 | 0.0707 | 0.0739 | ✓ beats it, −4.3% |
| **llff_fern** | 0.0367 | 0.0367 | 0.0381 | ✓ beats it, −3.7% |
| **llff_flower** | 0.0859 | 0.0863 | 0.0874 | ✓ beats it, −1.3% |
| **room** | 0.0290 | 0.0289 | 0.0285 | ✗ loses, +1.4% |

The offline prediction and the fresh GPU measurement agree to three decimal places on every scene. **Tuned confidence-guided fusion now beats pure VGGT geometry in 3 of 4 real scenes**, and the one loss shrank from 138% worse (pre-fix) to 1.4% worse. This is a genuine, largely-positive result on self-consistency — not a claim that fusion always wins, and not validated against any ground-truth accuracy measure.

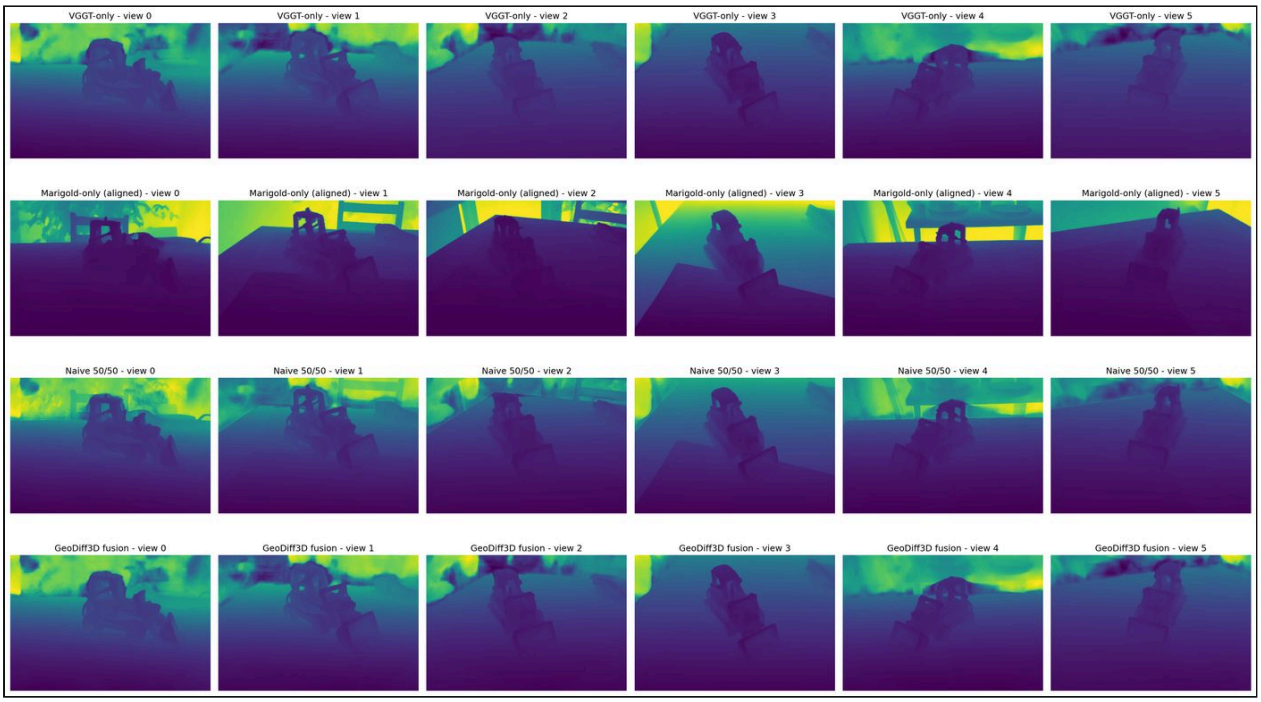


Fig. 1 — `kitchen`, the scene where fusion wins outright. All 6 views, all 4 methods. Fusion's background shows slightly more resolved structure than VGGT-only's own noisy background, not less — consistent with the capped correction adding real signal rather than just diluting toward Marigold.

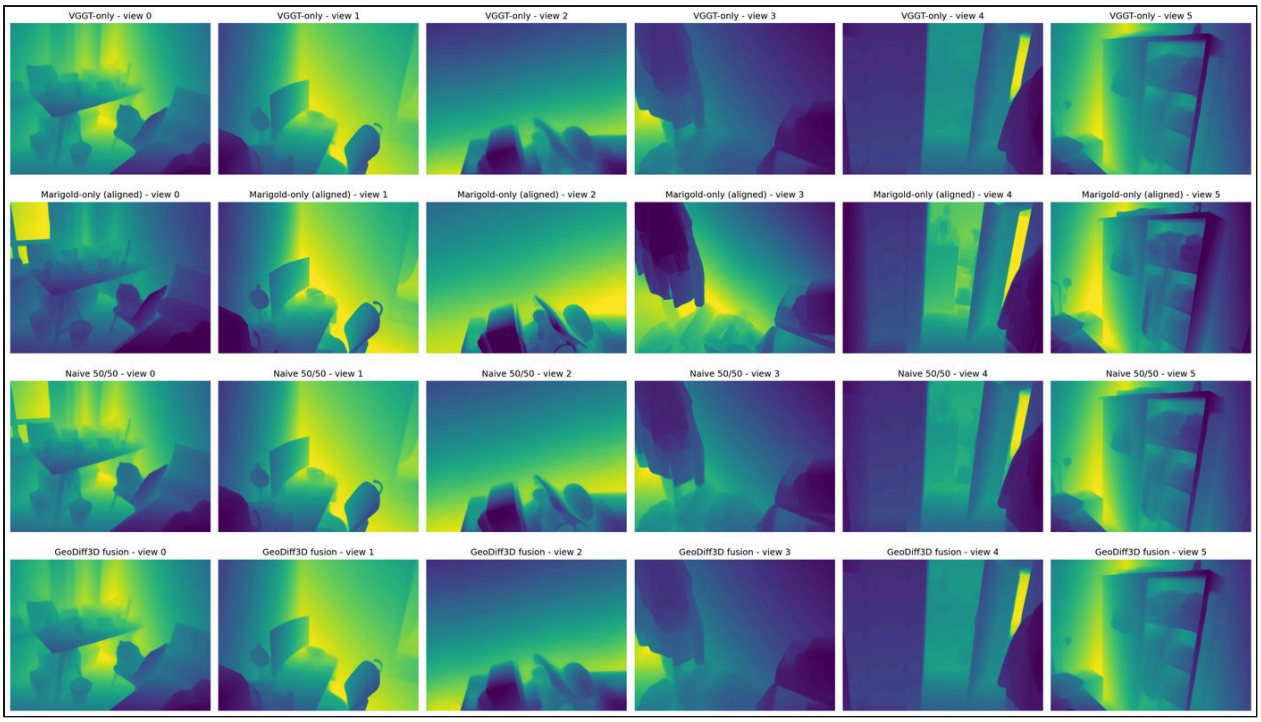


Fig. 2 — `room`, fusion's one loss. All four methods look visually near-identical here — a reminder that visual similarity doesn't imply comparable cross-view consistency; the metric is sensitive to differences these plots can't show.

## 08 Live verification, not just local numbers

Everything above ran through the same reproducible scripts. To confirm the whole system — frontend, backend, and a real GPU — actually works end-to-end as a deployed product, real photographs were uploaded through the production frontend, processed by the real backend on a Tesla T4 (via a temporary tunnel), and rendered back in the browser.

### What was uploaded

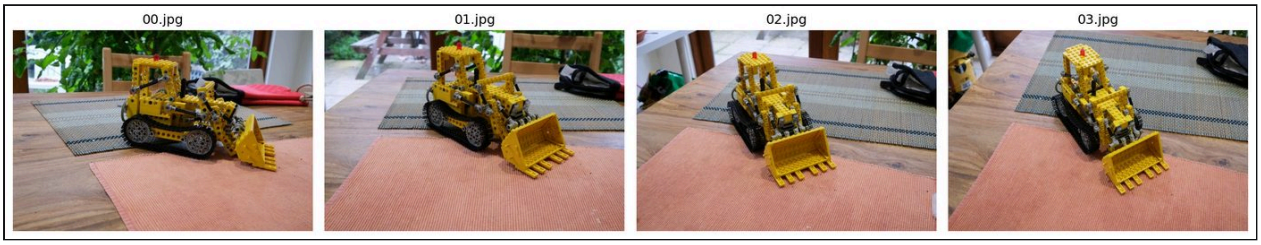


Fig. 3 — the four real photographs (a LEGO Technic excavator) uploaded through the live Reconstruct page for this run.

### What the live site rendered

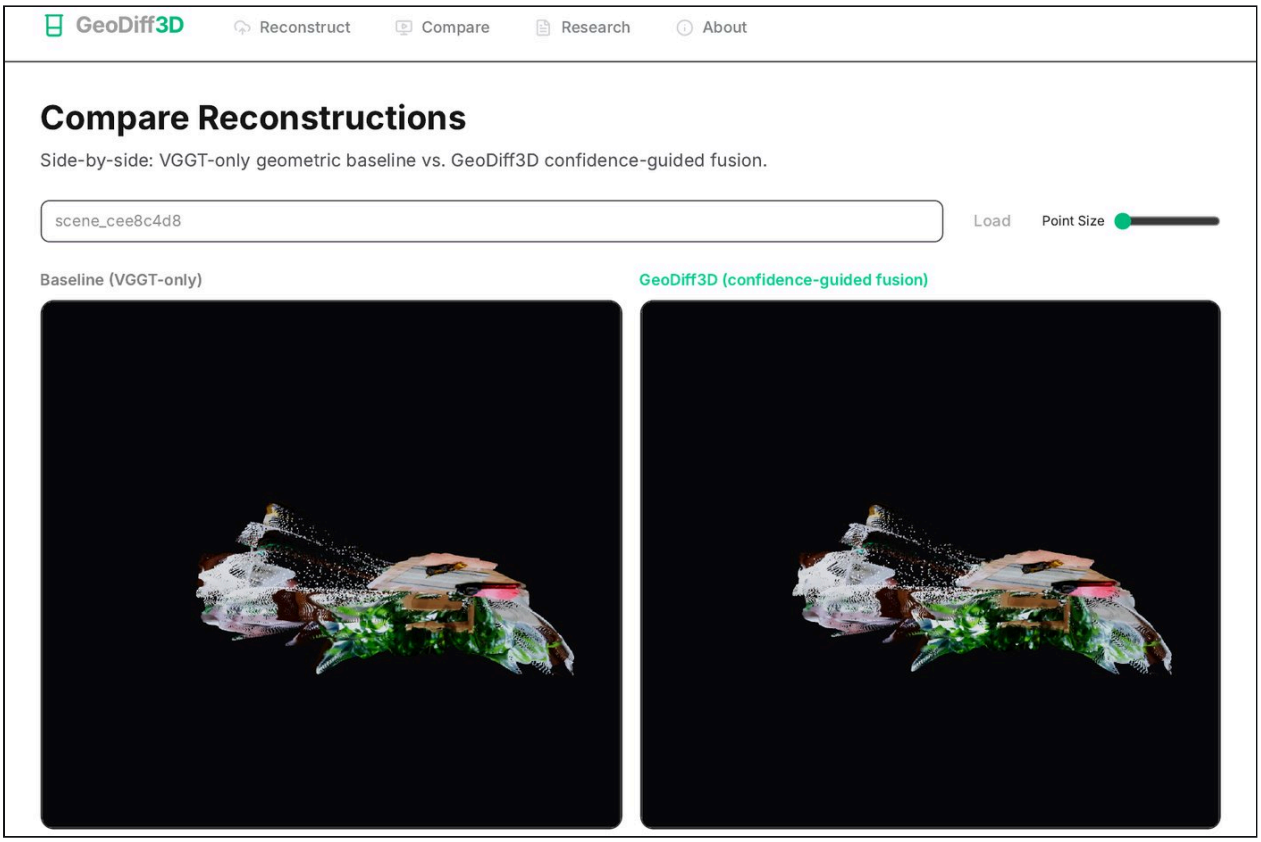


Fig. 5 — real screenshot, taken directly from the live public deployment, showing this exact scene's baseline and fusion point clouds rendered side by side on the Compare page.

### What the real depth maps looked like, this run

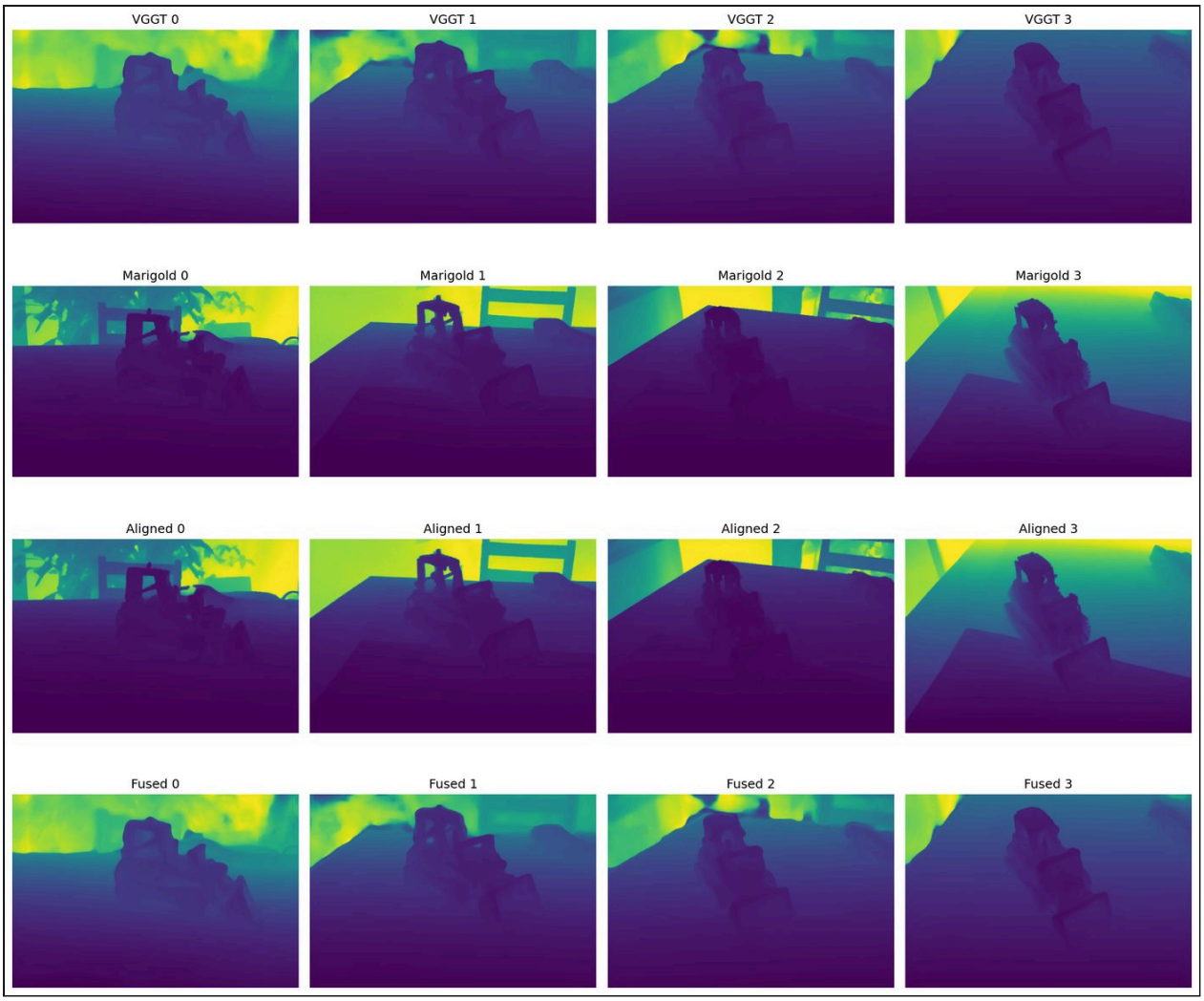


Fig. 4 — VGGT / Marigold / aligned / fused depth, generated live for this exact run — not a stock example.

## Server-side proof

The API responses below were captured directly from the live backend during this run:

```
GET /api/jobs/cee8c4d8-9d1c-4460-b3a6-f1701b39e5ed
state: completed

GET /api/scene/scene_cee8c4d8/download/metrics.json
vggt.model           facebook/VGGT-1B
vggt.device          cuda  (Tesla T4)
vggt.dtype           torch.float16
vggt.runtime_sec     2.677
marigold.model       prs-eth/marigold-depth-v1-1
marigold.runtime_sec 5.426
baseline.num_points  721,572
guided.num_points    721,572
baseline.depth_range 0.421 – 2.863
guided.depth_range   0.423 – 2.784
```

## 09 Engineering practice

A few things this project held itself to throughout, not just in the science:

- **No mocked results in production paths.** Every inference call either returns a real model output or raises — there is no synthetic fallback anywhere in the inference code.
- **20 commits, clean history, nothing squashed away.** Including the commit that shipped the buggy 0.4 default, and the separate commit that fixed it — the mistake is visible in the log, not edited out.
- **Every claim that changed was retracted in writing.** The 2-scene run's "fusion beats naive averaging" finding didn't hold at 4 scenes, and the results document says so explicitly rather than quietly updating the number.
- **Regression tests pin real behavior**, including a test that pins the exact shipped fusion defaults so a future change can't silently drift without updating the docs alongside it.
- **The frontend never fabricates data.** Metrics fields the real engine doesn't compute (AbsRel, RMSE, Chamfer, F-score) are absent from the UI entirely, not shown as blank or zero.

## 10 Limits, honestly

This is a self-consistency study, not an accuracy study — no claim here has been checked against a ground-truth scan, and the four scenes are all from VGGT's own example set, not an independent benchmark. The tuned parameters were chosen to optimize this specific metric on these four scenes; they may not generalize to very different scene types (crowds, dynamic scenes, extreme low-light). The live backend runs on a temporary Colab GPU with no uptime guarantee — it is a verification, not a hosted product.

Natural next steps: validate against a real ground-truth dataset (DTU, Tanks and Temples); expand past VGGT's own example photos to a broader, independently-sourced scene set; and investigate why the two outdoor/plant scenes responded differently to tuning than the indoor ones.

---